

- Q.1 과 Q.4 의 베릴로그 RTL에서 그래픽 장치를 다루는 방식의 차이점에 대하여 설명할 것

: Q01의 Space Invaders Verilog RTL을 보면 vga_x, vga_y라는 화면 좌표가 RTL의 입력으로 들어온다. 즉 현재 화면에서 어느 위치의 픽셀을 처리하고 있는지를 외부에서 알려주면, RTL에서는 그 좌표를 이용하여 현재 픽셀이 우주선, 총알, 외계인 등에 해당하는지를 판단한다.

반면 GLCD 방식에서는 RTL 내부에 scan_x, scan_y가 있고 이 좌표를 순서대로 변화시키면서 lcd_x, lcd_y로 출력, 따라서 GLCD에서는 RTL이 현재 그릴 화면의 좌표를 직접 만들어 디스플레이 장치로 전달한다.

처음에는 VGA와 GLCD의 차이가 단순히 화면의 해상도나 디스플레이 장치의 차이라고 생각했지만, 실제 RTL을 비교해보니 화면 좌표를 처리하는 방향 자체가 다르다는 점을 알 수 있었다.

Q01에서는 vga_x, vga_y를 받아서 픽셀을 결정하고, GLCD에서는 scan_x, scan_y를 이용하여 lcd_x, lcd_y를 만들어 출력한다. 즉,

VGA 방식은 외부에서 들어온 화면 좌표를 기준으로 픽셀을 만들어내는 방식,

GLCD 방식은 RTL 내부에서 화면 좌표를 만들어 디스플레이 장치에 전달하는 방식이라고 이해하였다.

같은 게임 그래픽을 표현하더라도 사용하는 그래픽 장치에 따라 RTL의 구조와 좌표 처리 방법이 달라질 수 있다는 것을 알게 되었다.

- Q.10을 실습 하십시오. 그리고 오류를 찾아내시기 바랍니다.

: Q10의 코드를 먼저 빌드하고 실행해 보았다. 빌드 자체는 정상적으로 완료되었지만 실행하는 과정에서 다음과 같은 오류가 발생.

libsystemc.so.3.0: cannot open shared object file

처음에는 코드에 문제가 있는 것으로 생각했지만, 확인해보니 실행 프로그램이 필요한 SystemC 공유 라이브러리의 위치를 찾지 못해서 발생한 오류였다.

SystemC 라이브러리가 설치되어 있는 경로를 실행 환경에 추가하기 위해 다음 명령을 사용하였다.

export LD_LIBRARY_PATH=/opt/systemc/lib:\$LD_LIBRARY_PATH

그 후 실행 파일을 다시 실행하였고 정상적으로 실행되는 것을 확인하였다.

이번 오류를 해결하면서 프로그램이 정상적으로 컴파일되는 것과 실제로 실행되는 것은 별개의 문제라는 것을 알게 되었다. 이번 오류는 Verilog RTL 자체의 컴파일 오류가 아니라 실행 단계에서 필요한 SystemC 라이브러리의 위치를 찾지 못해서 발생한 환경 설정 문제였다.

- Q.10 의 실습을 수행 할 것. 스프라이트 시현 방식에 대하여 아두이노 C++와 베릴로그 RTL의 차이를 비교 설명 할 것.

: Q10에서는 space_invaders_engine_glcd.v에 8x8 크기의 Sprite, Score 카운터와 폰트, Life 시스템을 추가한 코드를 확인하고 직접 빌드 및 실행하였다.

Sprite는 sprite_invader와 sprite_cannon이라는 배열에 8비트 데이터로 저장되어 있었다. 8비트 데이터를 한 줄이라고 생각하면 총 8개의 데이터가 모여 하나의 8x8 그림을 만드는 구조이다.

화면을 그릴 때는 현재 화면의 좌표인 scan_x, scan_y와 Sprite의 시작 위치인 curr_inv_x, curr_inv_y를 비교한다. 이 차이를 이용하여 현재 화면의 픽셀이 Sprite의 몇 번째 행과 열에 해당하는지를 찾고, 그 위치의 비트가 1이면 해당 픽셀을 화면에 표시한다.

Score는 score_bcd에 저장하고 font_rom에 저장된 숫자 모양을 이용하여 화면 위쪽에 점수를 표시한다. 또한 player_lives와 게임 상태를 이용하여 플레이어의 목숨과 게임오버 상태를 처리하고 있었다.

아두이노 C++에서 Sprite를 표시하는 경우에는 프로그램이 Sprite의 위치와 비트맵 데이터를 계산한 후 LCD에 그림을 그리는 함수를 호출하는 방식으로 생각할 수 있다. 즉 소프트웨어에서는 Sprite라는 그래픽 데이터를 하나의 객체처럼 다루면서 프로그램의 실행 순서에 따라 화면에 표시할 수 있다.

반면 Verilog RTL에서는 Sprite를 단순한 그림 객체로만 생각하는 것이 아니라 실제 하드웨어에서 처리할 수 있도록 비트 단위의 데이터로 저장한다. 그리고 현재 화면 좌표와 Sprite의 위치를 비교하여 현재 픽셀에 해당하는 Sprite의 비트가 1인지 0인지 판단하고 그 결과를 픽셀 출력에 연결한다.

따라서 아두이노 C++에서는 Sprite를 프로그램에서 처리하여 LCD에 출력하는 방식이라면, Verilog RTL에서는 Sprite의 각 비트를 하드웨어적으로 처리하여 화면의 픽셀을 만들어내는 방식이라고 이해하였다. 같은 Sprite를 표시하더라도 소프트웨어는 순차적인 프로그램의 관점에서 처리하고, RTL은 픽셀과 비트의 관계를 하드웨어 구조로 표현한다는 차이가 있었다.

- Q.11 의 실습을 수행 할 것. 좌우 화살표 키보드 입력에 대하여 이상 반응을 보이는 이유를 찾을 것.

: Q11 실습을 진행하면서 좌우 화살표 키를 눌렀을 때 우주선이 너무 빠르게 움직이는 문제가 있었다.

처음에는 키보드 입력을 받는 SystemC 코드에 문제가 있다고 생각했다. 하지만 이전 코드와 비교해 보니 키보드를 읽는 방법은 거의 같았다.

그 후 Verilog 코드를 비교해 보았다. 이전 코드에서는 game_tick이라는 조건을 이용하여 우주선의 이동 속도를 조절하고 있었지만, Q11에서는 좌우 이동 부분에서 이 조건이 빠져 있었다.

그래서 키를 누르고 있는 동안 우주선의 위치가 너무 자주 바뀌면서 빠르게 움직이는 문제가 발생한 것으로 확인했다. 그래서 키를 누르고 있는 동안 우주선의 위치가 너무 자주 바뀌면서 빠르게 움직이는 문제가 발생한 것으로 확인했다.

이번 실습을 통해 키보드 입력을 받는 부분과 실제 우주선을 움직이는 부분은 서로 다르기 때문에 두 부분의 코드를 따로 확인해야 한다는 것을 알게 되었다.

- Q.13의 시뮬레이션 디렉토리에서 실습을 실시할 것.-----Q13이 확인되지않아, 보류

* Q.11의 SystemC 테스트 벤치가 변경된 점을 찾아볼 것.

: Q09c와 Q11의 테스트벤치를 비교하였다.

두 코드 모두 SDL을 이용해서 왼쪽, 오른쪽, 스페이스 키를 확인하고 있었다.

Q11에서는 코드를 조금 더 간단하게 정리하고 필요하지 않은 주석과 출력문을 삭제하였다.

따라서 키보드를 입력받는 기본적인 방법은 Q09c와 Q11에서 크게 달라지지 않았다.

따라서 좌우 화살표의 이상 반응은 SystemC 테스트벤치보다는 Verilog RTL의 이동 처리 부분에서 발생한 것으로 판단하였다.

* 키보드 입력을 보완하기 위해 베릴로그 RTL의 차이점을 찾아볼 것

: Q11에서는 Q09b의 기본적인 좌우 이동 코드를 유지하면서 게임 상태를 STATE_START, STATE_PLAY, STATE_DIE, STATE_OVER로 확장하였다.

또한 목숨을 관리하는 player_lives와 피격 후 대기 시간을 관리하는 die_timer를 추가하였다.

좌우 키 입력은 STATE_PLAY 상태에서 btn_left, btn_right를 통해 cannon_x를 변경하는 방식으로 처리하였다.

Q09b에서는 좌우 이동이 game_tick 조건과 함께 동작했지만, Q11에서는 좌우 이동 부분에서 game_tick 조건이 빠져 있었다. 이 때문에 키를 누르고 있는 동안 우주선의 위치가 너무 자주 변경되어 빠르게 움직이는 문제가 발생하였다.

이 부분이 Q11에서 발생한 좌우 화살표 입력의 이상 반응과 관련된 중요한 차이라고 생각하였다.

- 자신만의 문장으로 이 둘(소프트웨어 언어와 하드웨어 언어)을 비교할 수 있도록 정리하기 바랍니다.

: 이번 실습을 하면서 소프트웨어 언어와 하드웨어 기술 언어는 단순히 사용하는 문법만 다른 것이 아니라 동작을 생각하는 방법 자체가 다르다는 것을 알게 됨.

C/C++ 같은 소프트웨어에서는 기본적으로 작성된 명령어를 순서대로 실행하면서 하나의 프로그램을 동작시킨다.

반면 Verilog와 같은 하드웨어 기술 언어에서는 여러 회로가 동시에 동작하기 때문에 프로그램의 실행 순서보다는 어떤 회로가 만들어지고 서로 어떻게 연결되는지를 생각해야 한다.

또한 하드웨어 언어로 작성한 알고리즘을 실제 칩으로 만들기 위해서는 합성 가능한 RTL 형태로 작성되어야 한다.

단순히 소프트웨어에서 사용하는 알고리즘을 Verilog 문법으로 바꾸는 것만으로는 충분하지 않고, 실제 레지스터와 조합회로가 어떻게 동작할지를 표현해야 한다.

하드웨어를 기술할 때는 다음과 같은 차이도 고려해야 한다.

비트 단위 객체: 하드웨어에서는 데이터를 단순한 변수로 생각하기보다는 실제 신호선이나 레지스터를 구성하는 비트 단위로 표현해야 한다.

다중값 논리: Verilog에서는 일반적인 0, 1뿐만 아니라 아직 값이 정해지지 않은 X와 고임피던스 상태를 나타내는 Z도 사용할 수 있다.

시간과 병렬성: 하드웨어에서는 여러 회로가 동시에 동작하므로 클럭과 시간에 따른 신호의 변화를 함께 생각해야 한다.

상태와 사건: FSM처럼 현재 상태에 따라 다음 동작이 결정되며, posedge와 같은 클럭의 예지나 이벤트를 기준으로 상태가 변화할 수 있다.

이번 실습에서도 플레이어 이동, 총알, 외계인, 게임 상태, 픽셀 출력 등을 하나의 프로그램처럼 순서대로 실행하는 것이 아니라 각각의 하드웨어 동작으로 구성해야 했다. 따라서 소프트웨어에서 생각한 알고리즘을 그대로 옮기는 것이 아니라 하드웨어의 병렬성, 시간, 상태 및 신호의 관계를 생각하면서 RTL로 다시 표현해야 한다는 점을 이해하게 됨.

- SystemC에 대한 이해를 높이도록 하십시오. 인공지능이 알려준 장점을 나름대로 정리하십시오

: SystemC는 처음에는 단순히 C++로 작성된 테스트벤치라고 생각했지만, 실습을 진행하면서 C++을 기반으로 하드웨어의 동작을 모델링하고 병렬적인 시뮬레이션을 할 수 있도록 만든 환경이라는 것을 알게 되었다.

Q09에서는 Verilator를 이용하여 Verilog RTL을 SystemC에서 사용할 수 있는 형태로 변환하였다. 이후 SystemC 테스트벤치에서 클럭과 리셋, 키보드 입력 등을 하드웨어에 전달하고 SDL2를 이용하여 게임 화면도 확인하였다.

SystemC의 장점은 C++의 장점을 이용하면서도 하드웨어의 병렬적인 동작을 표현할 수 있다는 것. 예를 들어 클럭을 발생시키는 동작과 키보드를 확인하는 동작, 그래픽을 처리하는 동작을 각각의 프로세스로 나누어 구성할 수 있다.

따라서 하드웨어의 여러 부분이 동시에 동작하는 모습을 C++ 환경에서 모델링하고 확인할 수 있다.

또한 Verilog RTL만으로 테스트하기 어려운 키보드나 그래픽 같은 기능을 C++과 SDL2를 이용하여 연결할 수 있다는 점도 실습하면서 느낀 장점이었다. 그래서 SystemC는 Verilog를 대신하는 언어라기보다는 C++의 구조와 기능을 이용하여 하드웨어 시스템을 모델링하고, 실제 하드웨어의 병렬적인 동작을 시뮬레이션하기 위한 환경이라고 이해하게 되었다.